Code Explanation

Data Processing Test Suite

This test suite is designed to validate the functionality of data processing tasks, including data cleaning, total amount calculation, and joining customer and order data. It utilizes PySpark for data processing and unittest for testing.

Overview
The code is a test suite written in Python using the unittest framework. It tests the functionality of three data processing functions: `clean_data`, `calculate_total_amount`, and `join_customer_order`. The tests are performed on sample customer and order data.

Step 1: Setting Up the Test Environment
This step involves creating a SparkSession and sample customer and order data for testing.

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder
        .appName("TestDataProcessing")
        .master("local[1]")
        .getOrCreate()

    # Create sample customer data
    customer_data = [
        ("C001", "John Doe", "john@example.com", "North"),
        ("C002", "Jane Smith", "jane@example.com", "South"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        ("C004", "Alice Brown", "alice@example.com", "West"),
        ("C005", None, "invalid@example.com", "North"),  # Contains
null
        ("C001", "John Doe", "john@example.com", "North")  # Duplica
te
    ]

    customer_schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    cls.customer_df = cls.spark.createDataFrame(customer_data, schem
a=customer_schema)
```

Step 2: Creating Sample Order Data
This step involves creating sample order data.

```
# Create sample order data
order_data = [
    ("O001", "Laptop", 1000.0, 2, datetime.date(2023, 1, 15), "C001"
),
    ("O002", "Phone", 500.0, 1, datetime.date(2023, 2, 20), "C002"),
    ("O003", "Tablet", 300.0, 3, datetime.date(2023, 3, 10), "C003")
```

```
,
    ("O004", "Monitor", 200.0, 2, datetime.date(2023, 4, 5), "C001")
,
    ("O005", "Keyboard", 50.0, None, datetime.date(2023, 5, 12), "C0
04"),  # Contains null
    ("O001", "Laptop", 1000.0, 2, datetime.date(2023, 1, 15), "C001"
)  # Duplicate
]

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])

cls.order_df = cls.spark.createDataFrame(order_data, schema=order_sc
hema)
```

Step 3: Testing Data Cleaning
This step involves testing the `clean_data` function on both customer and order data.

```
def test_clean_data(self):
    # Test cleaning customer data
    clean_customer_df = clean_data(self.customer_df)
    self.assertEqual(clean_customer_df.count(), 4)  # Should remove
null and duplicate

    # Test cleaning order data
    clean_order_df = clean_data(self.order_df)
    self.assertEqual(clean_order_df.count(), 4)  # Should remove nul
l and duplicate
```

Step 4: Testing Total Amount Calculation
This step involves testing the `calculate_total_amount` function.

```
def test_calculate_total_amount(self):
    # Test calculating total amount
    order_with_total = calculate_total_amount(self.order_df)

    # Check if TotalAmount column exists
    self.assertTrue("TotalAmount" in order_with_total.columns)

    # Check calculation for a specific row
    first_row = order_with_total.filter("OrderId = 'O001'").first()
    self.assertEqual(first_row["TotalAmount"], 2000.0)  # 1000.0 * 2
```

Step 5: Testing Joining Customer and Order Data
This step involves testing the `join_customer_order` function.

```
def test_join_customer_order(self):
    # Clean the data first
    clean_customer_df = clean_data(self.customer_df)
    clean_order_df = clean_data(self.order_df)

    # Calculate total amount
    order_with_total = calculate_total_amount(clean_order_df)

    # Join the data
    joined_df = join_customer_order(clean_customer_df, order_with_to
tal)

    # Check if join worked correctly
    self.assertEqual(joined_df.count(), 4)  # All valid records shou
ld be joined

    # Check if all required columns are present
    expected_columns = ["CustId", "Name", "EmailId", "Region", "Orde
rId",
                        "ItemName", "PricePerUnit", "Qty", "Date", "T
otalAmount"]
    self.assertEqual(set(joined_df.columns), set(expected_columns))

    # Check a specific joined record
    john_orders = joined_df.filter("Name = 'John Doe'").count()
    self.assertEqual(john_orders, 2)  # John should have 2 orders
```